Pinochle Multiplayer Mobile Game: Architecture Specification

1. High-Level Architecture

The system utilizes a "thin client, authoritative server" model to ensure competitive integrity, prevent cheating, and handle asynchronous mobile connectivity.

Client-Side (Mobile App)

- **Auth & Profile UI:** Handles login (UN/PW, Google Auth) and displays user statistics.
- **Matchmaking & Lobby UI:** Manages game creation/joining via 4-6 letters (no numbers) Game IDs and seat selection (North, East, South, West).
- **Game Board UI (Asset Manager):** The image-based rendering engine for the 48-card deck, avatars, and animations.
- **Network Synchronization Client:** Wraps REST API calls for out-of-game actions and manages the WebSocket connection for real-time game state updates.

Server-Side (Backend)

- **API Gateway / Auth Service:** Handles REST requests, validates tokens, and serves historical stats.
- **Lobby & Room Manager:** Generates unique Room IDs and manages concurrent seat selection.
- **WebSocket Pub/Sub Manager:** Maintains persistent TCP connections, routing client actions to game instances and broadcasting state updates.
- **Pinochle Game Engine (Source of Truth):** Validates all moves, calculates scores, limits information sent to clients (only sending a player's own hand), and drives the state machine.

Data Persistence

- **In-Memory Store (Redis):** Holds the active, real-time game state for lightning-fast reads/writes during active play.
- **Relational Database (PostgreSQL):** The permanent system of record for users, match history, deep analytics, and paused game states.

---

2. Database Schema (PostgreSQL)

The database is normalized to support deep statistical queries (e.g., win rates, trick-taking success, bid history) and allows games to be paused and resumed days later via State Hydration.

- **`users`**: Authentication and profiles.
  - `id` (UUID, PK), `username` (VARCHAR, Unique), `email` (VARCHAR, Unique, Nullable), `password_hash` (VARCHAR, Nullable), `google_auth_id` (VARCHAR, Unique, Nullable), `created_at` (TIMESTAMP), `updated_at` (TIMESTAMP), `deleted_at` (TIMESTAMP, Nullable for soft deletes).
- **`games`**: Tracks overall match data.
  - `id` (UUID, PK), `room_code` (VARCHAR(6)), `status` (ENUM: 'IN_PROGRESS', 'COMPLETED', 'ABANDONED'), `north_player_id`, `east_player_id`, `south_player_id`, `west_player_id` (UUID, FKs), `ns_total_score` (INT), `ew_total_score` (INT), `current_state_json` (JSONB - *Stores the active state to resume paused games later*), `started_at` (TIMESTAMP), `ended_at` (TIMESTAMP).
- **`hands`**: High-level summary of each round.
  - `id` (UUID, PK), `game_id` (UUID, FK), `hand_number` (INT), `winning_bidder_id` (UUID, FK), `winning_bid_amount` (INT), `is_shoot_the_moon` (BOOLEAN, Default False), `trump_suit` (ENUM), `ns_meld_score` (INT), `ew_meld_score` (INT), `ns_trick_score` (INT), `ew_trick_score` (INT), `is_set` (BOOLEAN).
- **`bids`**: Tracks the bidding war for a specific hand.
  - `id` (UUID, PK), `hand_id` (UUID, FK), `player_id` (UUID, FK), `bid_amount` (INT, Nullable for pass), `is_shoot_the_moon` (BOOLEAN, Default False), `bid_sequence` (INT).
- **`tricks`**: Tracks the individual 12 rounds of card-playing per hand.
  - `id` (UUID, PK), `hand_id` (UUID, FK), `trick_number` (INT 1-12), `led_by_player_id` (UUID, FK), `won_by_player_id` (UUID, FK), `north_card`, `east_card`, `south_card`, `west_card` (VARCHAR(2)), `trick_points` (INT).

---

3. Game Engine State Machine

The core loop ensuring all actions adhere to Pinochle rules. The state sits in Redis while active and flushes to Postgres (`current_state_json`) when players disconnect.

1. **`LOBBY_WAITING`**: Room created. Host can trigger `START_GAME` once all 4 seats are occupied.
2. **`DEALING`**: Server instantaneous state. Shuffles and assigns 12 cards per player.
3. **`BIDDING`**: Players submit bids, pass, or declare a "Shoot the Moon" bid. Ends when 3 players pass, or immediately if a player successfully bids to shoot the moon (depending on exact rule variations).
4. **`NAMING_TRUMP`**: Winning bidder declares the trump suit.
5. **`SHOWING_MELD`**: Server calculates and broadcasts all melds. Players must review and send an `ACKNOWLEDGE_MELD` action before gameplay begins.
6. **`TRICK_PLAYING`**: The core 12-trick loop. The server determines legal cards, awaits a valid `PLAY_CARD` action, and determines the trick winner.
7. **`HAND_SCORING`**: Server tallies points. If the hand was a "Shoot the Moon" bid, it applies the special win/loss condition. Otherwise, it checks if the bidding team was "set" against their numerical bid, and updates the game score. Transitions to `GAME_OVER` if a team hits the winning threshold, else loops to `DEALING`.
8. **`GAME_OVER`**: Match is complete. Permanent data is finalized in PostgreSQL.

---

4. API & WebSocket Contracts

REST API (Out-of-game)

- `POST /auth/login` | `POST /auth/google` -> Returns JWT.
- `GET /users/{user_id}/stats` -> Returns historical win/loss/trick records.
- `POST /games/create` -> Initializes a game, returns `{ room_id: "A7BX" }`.
- `POST /games/{room_id}/join` -> Validates room.

WebSocket Protocol (In-game JSON Payloads)

Lobby & Setup

- **Client -> Server:** `{"action": "SELECT_SEAT", "payload": {"seat": "NORTH"}}`
- **Client -> Server:** `{"action": "START_GAME", "payload": {}}` *(Host only)*
- **Server -> Client:** `{"event": "LOBBY_STATE_UPDATED", "payload": {"host_id": "...", "seats": {...}}}`

Dealing & Bidding

- **Server -> Client:** `{"event": "HAND_DEALT", "payload": {"cards": ["AH", "TH", ...]}}` *(Private)*
- **Server -> Client:** `{"event": "BIDDING_TURN", "payload": {"current_highest_bid": 25, "highest_bidder_seat": "NORTH", "next_to_act_seat": "EAST", "minimum_valid_bid": 26}}`
- **Client -> Server:** `{"action": "SUBMIT_BID", "payload": {"amount": 26, "shoot_the_moon": false}}`

Trump Naming & Meld

- **Client -> Server:** `{"action": "DECLARE_TRUMP", "payload": {"suit": "HEARTS"}}`
- **Server -> Client:** `{"event": "MELD_BROADCAST", "payload": {"trump_suit": "HEARTS", "winning_bid": 26, "is_shoot_the_moon": false, "bidding_team": "NS", "team_scores": {...}, "player_hands": {...}}}`
- **Client -> Server:** `{"action": "ACKNOWLEDGE_MELD", "payload": {}}`

Trick Playing

Server -> Client:

JSON

```
{
  "event": "YOUR_TURN",
  "payload": {
    "trick_number": 1,
    "led_suit": "SPADES",
    "currently_winning_card": "TS",
    "currently_winning_seat": "NORTH",
    "legal_cards": ["AS", "KS", "QS"]
  }
}
```

-
- **Client -> Server:** `{"action": "PLAY_CARD", "payload": {"card": "KS"}}`
- **Server -> Client:** `{"event": "CARD_PLAYED", "payload": {"seat": "SOUTH", "card": "KS"}}`
- **Server -> Client:** `{"event": "TRICK_COMPLETED", "payload": {"winner_seat": "NORTH", "trick_points_earned": 20, "ns_trick_score_total": 20, "ew_trick_score_total": 0, "next_to_lead": "NORTH"}}`

Scoring

- **Server -> Client:** json { "event": "HAND_COMPLETED", "payload": { "bidding_team": "NS", "bid_amount": 0, "is_shoot_the_moon": true, "ns_total_hand_points": 250, "ew_total_hand_points": 0, "bid_successful": true, "new_game_score": {"NS": 1500, "EW": 140}, "game_over": true } }

Redis State Hydration

When a game sits idle for too long (e.g., 30 minutes), we want to evict it from Redis to save server memory, relying completely on the `current_state_json` stored in Postgres. When a player opens the app a week later, we must seamlessly "wake" the game up.

Here is how we design the State Hydration logic.

1. The Hydration Flow (Sequence of Events)

When a client establishes a WebSocket connection and sends a `{"action": "RESUME_GAME", "payload": {"room_code": "A7BX"}}`:

1. **The Redis Check:** The WebSocket Manager asks Redis, "Do you have the state key for `game:A7BX`?"
2. **Cache Hit (Active Game):** If Redis says *yes*, the game is already awake (perhaps another player woke it up a few minutes ago). The server simply subscribes the user to the WebSocket room and sends them a state sync event.
3. **Cache Miss (Sleeping Game):** If Redis says *no*, the server initiates the Hydration protocol:
    - **Acquire Lock:** The server places a quick distributed lock in Redis (e.g., `SETNX lock:A7BX`). This is crucial. If two players open the app at the exact same millisecond, we only want *one* process querying Postgres and rebuilding the state.
    - **Fetch from DB:** The server queries Postgres: `SELECT current_state_json FROM games WHERE room_code = 'A7BX' AND status = 'IN_PROGRESS';`
    - **Hydrate Redis:** The server writes that JSON blob directly back into the `game:A7BX` key in Redis and sets a new Time-To-Live (TTL) of 30 minutes.
    - **Release Lock:** The lock is removed.
4. **Engine Initialization:** The Pinochle Game Engine instantiates a new state machine in memory, feeding it the hydrated JSON to set the exact phase, hands, and scores.
5. **Client Sync:** The server broadcasts a massive `STATE_RESTORED` payload back to the reconnecting client so their UI can draw the table exactly as they left it.

2. What's inside `current_state_json`?

To perfectly reconstruct the game, this JSON blob needs to capture a complete snapshot of the state machine. Every time a valid move is made during active gameplay, we fire off an async background task to update this JSON in Postgres.

Here is what it should look like:

JSON

```
{
  "room_code": "A7BX",
  "phase": "TRICK_PLAYING",
  "game_scores": {
    "NS": 850,
    "EW": 620
  },
  "current_hand": {
    "hand_number": 4,
    "dealer_seat": "EAST",
    "trump_suit": "SPADES",
    "bidding": {
      "winning_bid": 30,
      "winning_seat": "SOUTH",
      "is_shoot_the_moon": false
    },
    "team_meld": {
      "NS": 120,
      "EW": 40
```

```
    }
  },
  "player_hands": {
    "NORTH": ["AH", "TH", "KS"],
    "EAST": ["AD", "TD", "KD"],
    "SOUTH": ["AS", "TS", "QS"],
    "WEST": ["AC", "TC", "KC"]
  },
  "current_trick": {
    "trick_number": 10,
    "led_by_seat": "NORTH",
    "led_suit": "HEARTS",
    "next_to_act_seat": "EAST",
    "cards_on_table": {
      "NORTH": "AH",
      "EAST": null,
      "SOUTH": null,
      "WEST": null
    }
  }
}
```

3. The `STATE_RESTORED` WebSocket Event

When the client successfully wakes up the game, the server needs to send them all the information necessary to render the UI, *while maintaining our security rule of never exposing opponent's hidden cards*.

The server takes the hydrated state, scrubs the `player_hands` for the other three seats, determines the legal moves for the active player, and sends this to the reconnecting client:

Server -> Client (Private Sync Message):

JSON

```
{
  "event": "STATE_RESTORED",
  "payload": {
    "phase": "TRICK_PLAYING",
    "my_seat": "SOUTH",
    "scores": {"NS": 850, "EW": 620},
    "trump_suit": "SPADES",
    "my_hand": ["AS", "TS", "QS"],
    "cards_on_table": {
      "NORTH": "AH",
      "EAST": null,
      "SOUTH": null,
      "WEST": null
    },
    "is_my_turn": false,
    "next_to_act_seat": "EAST",
    "legal_cards": []
  }
}
```

By cleanly separating your ephemeral fast-state (Redis) from your permanent slow-state (Postgres) like this, your servers can restart, scale up or down, and crash without anyone ever losing a game in progress.

The Problem: The "Read-Modify-Write" Trap

Here is what happens if we build the lobby incorrectly:

1. **Thread A (Player 1)** asks Redis: "Is North empty?" -> Redis says YES.
2. **Thread B (Player 2)** asks Redis: "Is North empty?" -> Redis says YES.
3. **Thread A** writes Player 1 to North.
4. **Thread B** writes Player 2 to North, overwriting Player 1. *(Result: The UI shows Player 2 in North, but Player 1's client still thinks they are in North. Chaos ensues.)*

The Solution: Redis Atomic Commands (`HSETNX`)

Instead of asking Redis *if* the seat is empty before writing, we use an atomic command to say: **"Write my name into this seat ONLY IF it is currently empty."**

In Redis, this command is called `HSETNX` (Hash Set If Not Exists). Because Redis is single-threaded at its core, it processes commands sequentially. It is physically impossible for two `HSETNX` commands to execute simultaneously.

Here is the exact step-by-step logic when the server receives a `SELECT_SEAT` WebSocket action.

The Lobby Concurrency Flow

1. The Client Action arrives Both Player 1 and Player 2 send this payload at the exact same moment: `{"action": "SELECT_SEAT", "payload": {"seat": "NORTH"}}`

2. The Server Executes the Atomic Command The server routes both requests to Redis. Redis queues them up and processes them one by one, separated by microseconds.

- **For Player 1 (who arrived a microsecond faster):** The server runs: `HSETNX room:A7BX:seats NORTH player-uuid-1` Redis evaluates it: "North is empty. Writing player-uuid-1. Return `1` (Success)."
- **For Player 2 (who arrived a microsecond later):** The server runs: `HSETNX room:A7BX:seats NORTH player-uuid-2` Redis evaluates it: "North already has a value. Doing nothing. Return `0` (Failure)."

3. The Server Routes the Responses

Because the database handled the concurrency safely, our backend logic becomes incredibly simple. We just look at the integer returned by Redis.

If Redis returns `1` (Success): The server knows Player 1 successfully claimed the seat. It broadcasts the updated lobby to *everyone* in the room.

JSON

```
{
  "event": "LOBBY_STATE_UPDATED",
  "payload": {
    "seats": {"NORTH": "player-uuid-1", "EAST": null, "SOUTH": null, "WEST": null}
  }
}
```

If Redis returns `0` (Failure): The server knows Player 2 lost the race. It sends a private error event *only* to Player 2, allowing their client UI to display a quick "Seat already taken!" toast notification.

JSON

```
{
  "event": "SEAT_CLAIM_FAILED",
  "payload": {
    "message": "The North seat was claimed by another player.",
    "requested_seat": "NORTH"
  }
}
```